

Material de Estudos: Testes em Java

TechDesk Chamados API

Teoria e prática de cada fase, com código comentado a partir do seu próprio projeto

Setembro de 2026

Sumário

Material de Estudos: Testes em Java

TechDesk Chamados API

Como usar este material

Fase 1 — Testes unitários

1. Por que começar por testes unitários
2. JUnit 5 essencial
3. AssertJ: asserções fluentes
4. Mockito e test doubles
5. Corrigindo o teste quebrado do seu `CategoriaServiceTest`
6. TDD na prática: Red → Green → Refactor
7. Mutation testing com PIT: testando os seus testes
8. Exercício desta fase

Fontes oficiais

Fase 2 — Integração com Spring Boot

1. Por que integração é diferente
2. `@WebMvcTest` + `MockMvc`: testando só a camada web
3. `@DataJpaTest` + `Testcontainers`: persistência de verdade
4. Testando a segurança: JWT sem depender do banco de verdade
5. `ArchUnit`: travando a arquitetura em teste
6. Exercício desta fase

Fontes oficiais

Fase 3 — Automação de API

1. Integração vs. automação de API: qual a diferença
2. Projeto separado
3. `RestAssured`: request/response reutilizáveis
4. Autenticando o fluxo: login gera o token, o token vai nas próximas chamadas
5. Limpeza de dados
6. `Cucumber` (BDD): quando vale a pena
7. E a automação de UI (`Playwright`)?
8. Exercício desta fase

Fontes oficiais

Fase 4 — CI/CD, performance e IA

1. Por que colocar isso em CI
2. Pipeline no GitHub Actions com MySQL
3. `JaCoCo` com meta mínima
4. Teste de carga com `Gatling`
5. IA aplicada a testes: gerar, depois desconfiar
6. Aproveitando o módulo técnicos para praticar TDD completo

7. Exercício desta fase

Fontes oficiais

Fase 5 — Portfólio e entrevistas

1. O README é a sua vitrine
2. Currículo: bullets com verbo, ferramenta e resultado
3. Perguntas de entrevista baseadas no seu próprio projeto
4. Um post curto para o LinkedIn
5. Checklist final antes de aplicar para vagas

Sobre as fontes desta fase

Como usar este material

Este material transforma o plano de 12 semanas em um passo a passo de teoria e código comentado, sempre usando o seu projeto `techdesk-chamados-api` como exemplo.

- Siga a ordem das fases: Fase 1 → Fase 2 → Fase 3 → Fase 4 → Fase 5.
- Em cada fase: primeiro a teoria (curta), depois o código comentado, depois um exercício para aplicar no seu projeto.
- Não copie e cole os códigos sem entender: mude uma linha, rode de novo, veja o que muda.
- O cronograma completo (semanas, horas, entregáveis) está no documento "Plano de Estudos: Testes em Java (12 semanas)".

Fase	Foco
Fase 1	Testes unitários: JUnit 5, AssertJ, Mockito, TDD, mutation testing (PIT)
Fase 2	Integração: Spring Boot Test, MockMvc, Testcontainers, WireMock, ArchUnit
Fase 3	Automação de API: RestAssured, autenticação JWT nos testes
Fase 4	CI/CD, JaCoCo, performance (Gatling) e IA aplicada a testes
Fase 5	Portfólio, README, currículo e perguntas de entrevista

Fase 1 — Testes unitários

1. Por que começar por testes unitários

Um teste unitário testa uma classe **sozinha**, sem subir o Spring, com as dependências trocadas por mocks. Ele roda em milissegundos, então você pode rodar centenas deles a cada poucos minutos. É aqui que você prova que a **regra de negócio** está certa: cálculo, validação, decisão de fluxo.

2. JUnit 5 essencial

```
import org.junit.jupiter.api.*;
import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.ValueSource;

class ExemploJUnit5Test {

    @BeforeEach
    void setUp() {
        // roda antes de CADA teste desta classe – bom para recriar objetos e evitar
        // que um teste "suje" o estado do outro
    }

    @Test
    @DisplayName("Deve somar dois números positivos")
    void deveSomarDoisNumerosPositivos() {
        int resultado = 2 + 3;
        Assertions.assertEquals(5, resultado);
    }

    @Nested
    @DisplayName("Quando o valor é inválido")
    class QuandoValorInvalido {
        // agrupa testes relacionados; aparece "aninhado" no relatório, deixando
        // a intenção do grupo mais clara que um nome de método gigante
    }

    @ParameterizedTest
    @ValueSource(strings = {"", " ", "  "})
    @DisplayName("Deve rejeitar título em branco em qualquer uma destas variações")
    void deveRejeitarTituloEmBranco(String tituloInvalido) {
        Assertions.assertTrue(tituloInvalido.isBlank());
        // um só teste roda 3 vezes, uma por valor da lista – evita copiar e colar
        // o mesmo teste só mudando o valor de entrada
    }
}
```

Padrão Arrange-Act-Assert (AAA): organize sempre em três blocos — monta os dados (Arrange), chama o método (Act), confere o resultado (Assert). Isso por si só já deixa o teste mais fácil de ler.

3. AssertJ: asserções fluentes

AssertJ lê quase como português e dá mensagens de erro muito melhores que o `Assertions` puro do JUnit, principalmente em coleções e objetos.

```
import static org.assertj.core.api.Assertions.assertThat;

// comparando um objeto inteiro de uma vez, campo a campo
assertThat(categoriaResponse)
    .extracting("id", "nome")
    .containsExactly(1L, "Hardware");

// coleções: tamanho + conteúdo em uma linha
assertThat(chamados)
    .hasSize(2)
    .extracting("titulo")
    .containsExactlyInAnyOrder("Site com erro 500", "Windows não abre");

// exceções: verifica o tipo E a mensagem
assertThat(catchThrowable(() -> categoriaService.buscar(999L)))
    .assertInstanceOf(CategoriaNaoEncontradaException.class)
    .hasMessageContaining("999");
```

4. Mockito e test doubles

Tipo	O que faz	Exemplo no seu projeto
Mock	objeto falso, você programa cada resposta	<code>CategoriaRepository</code> mockado no teste do <code>CategoriaService</code>
Stub	uma resposta fixa, sem verificar chamadas	um repositório que sempre devolve a mesma lista
Spy	objeto real, mas você pode espionar/sobrescrever parte dele	raro no dia a dia; use com cuidado
Fake	uma implementação simplificada que funciona de verdade	um repositório em memória, por exemplo

Regra prática: mocke as **fronteiras** (repository, cliente HTTP, serviço externo). Não mocke a classe que você está testando, nem objetos de valor simples como um DTO.

```

import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.InjectMocks;
import org.mockito.Mock;
import org.mockito.junit.jupiter.MockitoExtension;

import java.util.Optional;

import static org.assertj.core.api.Assertions.assertThat;
import static org.junit.jupiter.api.Assertions.assertThrows;
import static org.mockito.Mockito.*;

@ExtendWith(MockitoExtension.class) // liga o Mockito, SEM subir o Spring
class CategoriaServiceExemploTest {

    @Mock
    private CategoriaRepository categoriaRepository; // dependência falsa

    @Mock
    private ChamadoRepository chamadoRepository;

    @InjectMocks
    private CategoriaService categoriaService; // classe real, com os mocks injetados

    @Test
    void deveCriarCategoriaQuandoNomeNaoExiste() throws Throwable {
        // Arrange: o repositório ainda não tem essa categoria
        when(categoriaRepository.findByNome("Hardware")).thenReturn(Optional.empty());
        when(categoriaRepository.save(any(Categoria.class)))
            .thenAnswer(inv -> inv.getArgument(0)); // devolve o mesmo objeto que recebeu

        // Act
        var resultado = categoriaService.criar(new CategoriaRequestDto("Hardware"));

        // Assert: resultado E as interações com o mock
        assertThat(resultado.nome()).isEqualTo("Hardware");
        verify(categoriaRepository).findByNome("Hardware");
        verify(categoriaRepository).save(any(Categoria.class));
    }

    @Test
    void deveLancarExcecaoQuandoCategoriaJaExiste() {
        var existente = new Categoria(1L, "Eletrônicos", null);
        when(categoriaRepository.findByNome("Eletrônicos")).thenReturn(Optional.of(existente));

        assertThrows(CategoriaJaExisteException.class,
            () -> categoriaService.criar(new CategoriaRequestDto("Eletrônicos")));

        verify(categoriaRepository, never()).save(any()); // garante que NÃO tentou salvar
    }
}

```

5. Corrigindo o teste quebrado do seu `CategoriaServiceTest`

No diagnóstico do seu projeto, este teste chamou atenção:

```
// COMO ESTÁ – não testa nada de verdade
@Test
@DisplayName("Deve retornar uma lista vazia")
public void deveRetornarUmaListaVazia() {
    Mockito.when(categoriaService.findAll()).thenReturn(List.of());
    // categoriaService é a classe REAL sob teste (@InjectMocks), não um mock.
    // "when" aqui na verdade EXECUTA findAll() de verdade, que chama
    // categoriaRepository.findAll() – e como esse mock não foi programado,
    // ele devolve null, o que pode até quebrar o teste por acaso.

    List<CategoriaResponseDTO> allCategory = categoriaService.findAll();

    assertTrue(allCategory.isEmpty());
    Mockito.verify(categoriaRepository).findAll();
}
```

```
// CORRIGIDO – mocka a dependência (o repository), não a classe sob teste
@Test
@DisplayName("Deve retornar uma lista vazia quando não há categorias")
void deveRetornarUmaListaVaziaQuandoNaoHaCategorias() {
    when(categoriaRepository.findAll()).thenReturn(List.of()); // agora é o mock certo

    List<CategoriaResponseDTO> allCategory = categoriaService.findAll();

    assertThat(allCategory).isEmpty();
    verify(categoriaRepository).findAll();
}
```

Lição: `when(...)` só faz sentido em cima de um objeto que é um **mock**. Se o `when` está em cima da classe que o `@Test` deveria estar testando, o teste não está testando nada — ele só está chamando o método de verdade escondido dentro de um `when`.

6. TDD na prática: Red → Green → Refactor

Vamos adicionar uma regra nova ao `ChamadoService`: **não permitir reabrir um chamado já FECHADO sem um motivo**. Fazendo por TDD:

Red — escreva o teste antes de existir a regra (ele falha):

```

@Test
void naoDeveReabrirChamadoFechadoSemMotivo() {
    Chamado fechado = new Chamado(1L, "Impressora", "não imprime", categoria);
    fechado.setStatusChamado(StatusChamado.FECHADO);
    when(chamadoRepository.findById(1L)).thenReturn(Optional.of(fechado));

    assertThrows(ReaberturaSemMotivoException.class,
        () -> chamadoService.atualizarStatus(1L, StatusChamado.ABERTO));
    // este teste NEM COMPILA ainda: ReaberturaSemMotivoException não existe.
    // Isso é esperado no passo Red.
}

```

Green — o mínimo de código para o teste passar:

```

public ChamadoResponseDTO atualizarStatus(Long id, StatusChamado novoStatus) throws Throwable {
    Chamado chamado = chamadoRepository.findById(id)
        .orElseThrow(() -> new ChamadoNaoEncontradoException(id.toString()));

    if (chamado.getStatusChamado() == StatusChamado.FECHADO && novoStatus == StatusChamado.ABERTO)
        throw new ReaberturaSemMotivoException(id); // classe nova, mínima, só para passar
    }

    chamado.setStatusChamado(novoStatus);
    if (novoStatus == StatusChamado.FECHADO) {
        chamado.setDataFechamento(LocalDate.now());
    }
    return toResponseDTO(chamadoRepository.save(chamado));
}

```

Refactor — limpe sem mudar o comportamento, por exemplo extraindo a condição para um método privado `precisaDeMotivoParaReabrir(chamado, novoStatus)`. Rode os testes de novo a cada mudança: eles precisam continuar verdes.

7. Mutation testing com PIT: testando os seus testes

Cobertura de linha só diz que a linha **rodou**, não que o teste verificaria um erro nela. O PIT muda pequenos detalhes do seu código (um `>` vira `>=`, um `+` vira `-`) e roda seus testes: se **nenhum teste falhar**, aquele "mutante" sobreviveu — sinal de que a linha não está bem testada.

Adicione ao `pom.xml` (dentro de `<build><plugins>`):

```

<plugin>
  <groupId>org.pitest</groupId>
  <artifactId>pitest-maven</artifactId>
  <version>LATEST</version>
  <configuration>
    <targetClasses>
      <param>com.techdesk.techdesk.categorias.service.*</param>
      <param>com.techdesk.techdesk.chamados.service.*</param>
    </targetClasses>
    <targetTests>
      <param>com.techdesk.techdesk.*</param>
    </targetTests>
  </configuration>
</plugin>

```

Rode com `./mvnw org.pitest:pitest-maven:mutationCoverage` e abra o relatório HTML gerado em `target/pit-reports`. Para cada mutante sobrevivente, pergunte-se: *qual asserção eu preciso adicionar para matar esse mutante?*

8. Exercício desta fase

- ☐ Corrigir `deveRetornarUmaListaVazia` em `CategoriaServiceTest`.
- ☐ Escrever testes unitários para `ChamadoService`: `criar`, `criarEmLote`, `listarTodos`, `buscarPorId`, `atualizarStatus`, `atualizaChamado`, `excluir`, `deletarEmLote`.
- ☐ Rodar PIT sobre `CategoriaService` e `ChamadoService` e corrigir ao menos 3 mutantes sobreviventes.
- ☐ (Opcional, TDD) Implementar a regra `ReaberturaSemMotivoException` seguindo Red-Green-Refactor.

Fontes oficiais

- [JUnit 5 — User Guide](#)
- [AssertJ — documentação oficial](#)
- [Mockito — javadoc oficial \(mockito-core\)](#)
- [PIT \(pitest\) — Quickstart Maven](#)

Fase 2 — Integração com Spring Boot

1. Por que integração é diferente

Aqui você prova que as peças **conversam direito**: o controller devolve o JSON certo, o repositório gera o SQL certo, o filtro de segurança deixa passar quem deve. É mais lento que o unitário porque sobe pedaços do Spring, então você usa menos testes, mas cada um vale mais.

2. @WebMvcTest + MockMvc: testando só a camada web

Sobe só o `ChamadoController` (e o que ele precisa: conversão JSON, validação, `GlobalExceptionHandler`), com o `ChamadoService` mockado. Rápido, e testa exatamente o contrato HTTP.

```

import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.web.servlet.WebMvcTest;
import org.springframework.test.context.bean.override.mockito.MockitoBean;
import org.springframework.http.MediaType;
import org.springframework.test.web.servlet.MockMvc;

import static org.mockito.Mockito.when;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.*;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.*;

@WebMvcTest(ChamadoController.class) // sobe SÓ essa fatia da aplicação
class ChamadoControllerTest {

    @Autowired
    private MockMvc mockMvc;

    @MockitoBean // substitui o @MockBean (deprecated); disponível desde Spring Framework 6.1
    private ChamadoService service;

    @Test
    void deveRetornar201AoCriarChamado() throws Exception {
        var resposta = new ChamadoResponseDTO(1L, "Impressora", "não imprime",
            StatusChamado.ABERTO, "Manutenção", LocalDateTime.now());
        when(service.criar(any())).thenReturn(resposta);

        mockMvc.perform(post("/api/chamado")
            .contentType(MediaType.APPLICATION_JSON)
            .content("""
                {"titulo":"Impressora","descricao":"não imprime","categoriaId":1}
                """))
            .andExpect(status().isCreated())
            .andExpect(jsonPath("$.titulo").value("Impressora"));
    }

    @Test
    void deveRetornar404QuandoChamadoNaoExiste() throws Exception {
        when(service.buscarPorId(999L)).thenThrow(new ChamadoNaoEncontradoException("999"));

        mockMvc.perform(get("/api/chamado/999"))
            .andExpect(status().isNotFound()); // depende do @ControllerAdvice mapear certo
    }
}

```

Achado do seu projeto: `atualizaChamado` no `ChamadoController` devolve `HttpStatus.CREATED` (201) para um `PATCH`. Escreva o teste esperando o status **real** (201, hoje), veja passar, documente o problema e depois decida se corrige para 200 — um teste nunca deve "fingir" que o bug não existe.

3. @DataJpaTest + Testcontainers: persistência de verdade

Seu projeto usa **MySQL**, não H2. Se você testar com H2, pode passar no teste e falhar em produção por causa de diferenças de dialeto SQL. O Testcontainers sobe um MySQL real em Docker só para o teste.

```
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.orm.jpa.DataJpaTest;
import org.springframework.boot.testcontainers.service.connection.ServiceConnection;
import org.testcontainers.containers.MySQLContainer;
import org.testcontainers.junit.jupiter.Container;
import org.testcontainers.junit.jupiter.Testcontainers;

import static org.assertj.core.api.Assertions.assertThat;

@DataJpaTest
@Testcontainers
class CategoriaRepositoryIT {

    @Container
    @ServiceConnection // Spring Boot 3.1+: configura a datasource sozinho, sem @DynamicPropertySource
    static MySQLContainer<?> mysql = new MySQLContainer<>("mysql:8.0");

    @Autowired
    private CategoriaRepository categoriaRepository;

    @Test
    void deveEncontrarCategoriaPeloNome() {
        categoriaRepository.save(new Categoria(null, "Rede", null));

        var encontrada = categoriaRepository.findByNome("Rede");

        assertThat(encontrada).isPresent();
        assertThat(encontrada.get().getNome()).isEqualTo("Rede");
        // isto testa a query gerada pelo Spring Data contra um MySQL real
    }
}
```

Se o seu Spring Boot for anterior à 4.0/3.1, troque `@ServiceConnection` por um `@DynamicPropertySource` apontando `spring.datasource.url/username/password` para `mysql.getJdbcUrl()` etc.

4. Testando a segurança: JWT sem depender do banco de verdade

O fluxo do seu projeto é: `AuthController` → `AuthService` (autentica e gera token com `TokenService`) → `SecurityFilter` (lê o header `Authorization` em cada requisição seguinte).

```

import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.InjectMocks;
import org.mockito.Mock;
import org.mockito.junit.jupiter.MockitoExtension;
import org.springframework.security.authentication.AuthenticationManager;
import org.springframework.security.authentication.BadCredentialsException;

import static org.assertj.core.api.Assertions.assertThat;
import static org.junit.jupiter.api.Assertions.assertThrows;
import static org.mockito.Mockito.when;

@ExtendWith(MockitoExtension.class)
class AuthServiceTest {

    @Mock private AuthenticationManager authenticationManager;
    @Mock private UsuarioRepository userRepository;
    @Mock private TokenService jwtService;
    @Mock private PasswordEncoder passwordEncoder;
    @InjectMocks private AuthService authService;

    @Test
    void deveGerarTokenQuandoCredenciaisValidas() {
        var usuario = new Usuario("joao", "hash", UserRole.USER);
        var authentication = new UsernamePasswordAuthenticationToken(usuario, null);
        when(authenticationManager.authenticate(any())).thenReturn(authentication);
        when(jwtService.generateToken(usuario)).thenReturn("token-fake");

        var resposta = authService.login(new LoginRequestDTO("joao", "senha123"));

        assertThat(resposta.token()).isEqualTo("token-fake");
    }

    @Test
    void deveRejeitarLoginComCredenciaisInvalidas() {
        when(authenticationManager.authenticate(any()))
            .thenThrow(new BadCredentialsException("credenciais inválidas"));

        assertThrows(BadCredentialsException.class,
            () -> authService.login(new LoginRequestDTO("joao", "errada")));
    }

    @Test
    void deveRejeitarRegistroDeLoginJaExistente() {
        when(userRepository.existsByLogin("joao")).thenReturn(true);

        assertThrows(IllegalArgumentException.class,
            () -> authService.register(new RegisterRequest("joao", "senha123")));
    }
}

```

Para testar endpoints protegidos com `@WebMvcTest`, use `spring-security-test` e simule o usuário autenticado sem precisar de um token real:

```
import static org.springframework.security.test.web.servlet.request.SecurityMockMvcRequestPosters;

mockMvc.perform(get("/api/chamado").with(user("joao").roles("USER"))
    .andExpect(status().isOk());

mockMvc.perform(get("/api/chamado")) // sem autenticação
    .andExpect(status().isUnauthorized());
```

5. ArchUnit: travando a arquitetura em teste

O diagnóstico encontrou o `CategoriaService` recebendo `TechdeskChamadosApiApplication` no construtor, sem usar — um acoplamento que não deveria existir. O ArchUnit deixa essas regras escritas como teste, para nunca mais acontecer de novo. Dependência atual:

`com.tngtech.archunit:archunit:1.5.0`.

```
import com.tngtech.archunit.core.importer.ClassFileImporter;
import com.tngtech.archunit.lang.syntax.ArchRuleDefinition;

class ArquiteturaTest {

    @Test
    void servicesNaoDevemDependerDaClasseDeAplicacao() {
        var classes = new ClassFileImporter().importPackages("com.techdesk.techdesk");

        ArchRuleDefinition.noClasses()
            .that().resideInAPackage("..service..")
            .should().dependOnClassesThat().haveSimpleName("TechdeskChamadosApiApplication")
            .check(classes);
    }

    @Test
    void controllersNaoDevemAcessarRepositoriesDiretamente() {
        var classes = new ClassFileImporter().importPackages("com.techdesk.techdesk");

        ArchRuleDefinition.noClasses()
            .that().resideInAPackage("..controller..")
            .should().dependOnClassesThat().resideInAPackage("..repository..")
            .check(classes);
    }
}
```

6. Exercício desta fase

- ☐ `@WebMvcTest` para `ChamadoController` e `CategoriaController`: caminho feliz, 404, 400 (validação) e os dois casos de PATCH com status suspeito.
- ☐ `@DataJpaTest` + Testcontainers (MySQL) para `ChamadoRepository` e `CategoriaRepository`.
- ☐ Testes de `AuthService`: login válido, senha errada, registro duplicado.
- ☐ Um teste de endpoint protegido sem token (`401`) e com token/usuário simulado (`200`).
- ☐ Duas regras de ArchUnit, incluindo a do `TechdeskChamadosApiApplication` no `CategoriaService`.

Fontes oficiais

- [Spring Boot — Testing Spring Boot Applications](#) — confirma `@MockitoBean` como substituto oficial de `@MockBean` (Spring Framework 6.1+ / Spring Boot 3.2+) e o funcionamento de `@WebMvcTest`.
- [Spring Boot — Testcontainers](#) — uso de `@ServiceConnection`, disponível desde o Spring Boot 3.1.
- [Testcontainers — MySQL Module](#)
- [WireMock — Plain Java usage](#)
- [ArchUnit — Getting Started](#)

Fase 3 — Automação de API

1. Integração vs. automação de API: qual a diferença

Os testes da Fase 2 rodam **dentro** do processo Java, com Spring de teste. Automação de API testa a aplicação **de fora**, como um cliente HTTP real, contra uma instância rodando de verdade (local, staging etc.). É o mesmo tipo de teste que uma vaga de QA Automation/SDet cobra.

2. Projeto separado

Crie uma pasta `automation-tests/` ao lado do seu projeto, com seu próprio `pom.xml` (módulo Maven independente). Isso deixa claro que esses testes não fazem parte da aplicação, e podem rodar contra qualquer ambiente só trocando uma URL.

```
<dependency>
  <groupId>io.rest-assured</groupId>
  <artifactId>rest-assured</artifactId>
  <version>5.5.0</version>
  <scope>test</scope>
</dependency>
```

3. RestAssured: request/response reutilizáveis

```
import io.restassured.builder.RequestSpecBuilder;
import io.restassured.specification.RequestSpecification;
import org.junit.jupiter.api.BeforeAll;

class ConfigBase {

    static RequestSpecification spec;

    @BeforeAll
    static void setup() {
        // BASE_URL vem de variável de ambiente: local, CI ou staging usam a mesma suíte
        String baseUrl = System.getProperty("BASE_URL", "http://localhost:8080");

        spec = new RequestSpecBuilder()
            .setBaseUrl(baseUrl)
            .setContentType("application/json")
            .build();
    }
}
```

4. Autenticando o fluxo: login gera o token, o token vai nas próximas chamadas

```

import io.restassured.response.Response;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;

import static io.restassured.RestAssured.given;
import static org.hamcrest.Matchers.*;

class ChamadoApiTest extends ConfigBase {

    private String token;

    @BeforeEach
    void login() {
        // Arrange: usuário de teste já cadastrado no ambiente (ou registrado aqui mesmo)
        Response resposta = given(spec)
            .body("""
                {"login":"usuario.teste","senha":"senha123"}
                """)
            .when().post("/api/auth/login");

        token = resposta.jsonPath().getString("token");
    }

    @Test
    void deveCriarECriarChamadoComSucesso() {
        given(spec)
            .header("Authorization", "Bearer " + token)
            .body("""
                {"titulo":"Monitor piscando","descricao":"tela pisca ao ligar","categoriaId":1}
                """)
            .when()
            .post("/api/chamado")
            .then()
            .statusCode(201)
            .body("titulo", equalTo("Monitor piscando"))
            .body("status", equalTo("ABERTO"));
    }

    @Test
    void deveRejeitarRequisicaoSemToken() {
        given(spec)
            // sem o header Authorization de propósito
            .when()
            .get("/api/chamado")
            .then()
            .statusCode(401);
    }

    @Test
    void deveRetornar404AoBuscarChamadoInexistente() {
        given(spec)
            .header("Authorization", "Bearer " + token)

```

```
.when()  
  .get("/api/chamado/999999")  
  .then()  
    .statusCode(404);  
}  
}
```

5. Limpeza de dados

Cada teste que cria dados deve apagar o que criou (ou usar um endpoint/rotina de limpeza no `@AfterEach`), senão a suíte fica dependente de ordem e de execuções anteriores — uma das causas mais comuns de teste instável (flaky).

6. Cucumber (BDD): quando vale a pena

BDD ajuda quando o time de negócio precisa ler o cenário. Para uma API interna sem esse público, muitas vezes é só camada extra. Ainda assim, vale praticar, porque é muito pedido em vagas de QA:

```
# src/test/resources/features/chamado.feature  
Funcionalidade: Abertura de chamado  
  
Cenário: Usuário autenticado abre um chamado válido  
  Dado que o usuário "usuario.teste" está autenticado  
  Quando ele cria um chamado com titulo "Monitor piscando" na categoria "Manutenção"  
  Então o chamado é criado com status "ABERTO"
```

```

import io.cucumber.java.pt.Dado;
import io.cucumber.java.pt.Quando;
import io.cucumber.java.pt.Entao;

public class ChamadoSteps {

    private String token;
    private Response resposta;

    @Dado("que o usuario {string} esta autenticado")
    public void queOUusuarioEstaAutenticado(String login) {
        // reaproveita o mesmo cliente/spec da Seção 3
    }

    @Quando("ele cria um chamado com titulo {string} na categoria {string}")
    public void eleCriaUmChamado(String titulo, String categoria) {
        // monta o request e guarda a resposta em `resposta`
    }

    @Entao("o chamado e criado com status {string}")
    public void oChamadoECriadoComStatus(String status) {
        // assertThat(resposta.jsonPath().getString("status")).isEqualTo(status);
    }
}

```

7. E a automação de UI (Playwright)?

O `techdesk-chamados-api` é só uma API, sem tela própria. Não invente uma UI só para praticar Playwright aqui: pratique-o em outro exercício (um site de demonstração público) e mantenha este projeto focado em API. Se um dia o projeto ganhar um front-end ou uma tela via Swagger UI, essa seção volta a fazer sentido.

8. Exercício desta fase

- ☐ Módulo `automation-tests/` separado, com `BASE_URL` configurável.
- ☐ Suíte RestAssured cobrindo `/api/auth/login`, `/api/auth/register`, `/api/categoria/**` e `/api/chamado/**`, incluindo sucesso, 401, 404 e 400 (validação).
- ☐ Limpeza de dados após os testes que criam registros.
- ☐ Ao menos uma feature em Cucumber para o fluxo de abertura de chamado.

Fontes oficiais

- [REST Assured — Getting Started \(wiki oficial\)](#)
- [REST Assured — Docs](#)
- [Cucumber-JVM — repositório e documentação oficial](#)

Fase 4 — CI/CD, performance e IA

1. Por que colocar isso em CI

Teste que só roda na sua máquina não protege ninguém. O objetivo aqui é que toda vez que você (ou alguém) suba código, os testes rodem sozinhos e barrem o merge se algo quebrar.

2. Pipeline no GitHub Actions com MySQL

```

# .github/workflows/ci.yml
name: CI

on:
  push:
    branches: [main, estudos/testes]
  pull_request:

jobs:
  test:
    runs-on: ubuntu-latest

    services:
      mysql:
        image: mysql:8.0
        env:
          MYSQL_ROOT_PASSWORD: root
          MYSQL_DATABASE: techdesk_chamados
        ports:
          - 3306:3306
        options: >-
          --health-cmd="mysqladmin ping"
          --health-interval=5s
          --health-timeout=5s
          --health-retries=10

    steps:
      - uses: actions/checkout@v4

      - name: Configurar Java 21
        uses: actions/setup-java@v4
        with:
          distribution: temurin
          java-version: '21'
          cache: maven

      - name: Rodar testes unitarios e de integracao
        run: ./mvnw -B verify
        env:
          SPRING_DATASOURCE_URL: jdbc:mysql://localhost:3306/techdesk_chamados
          SPRING_DATASOURCE_USERNAME: root
          SPRING_DATASOURCE_PASSWORD: root

      - name: Publicar relatorio JaCoCo
        if: always()
        uses: actions/upload-artifact@v4
        with:
          name: jacoco-report
          path: target/site/jacoco

```

Se seus testes de integração já usam Testcontainers (Fase 2), você nem precisa do serviço `mysql` acima: o próprio Testcontainers sobe o banco dentro do runner. Escolha uma abordagem e documente a razão no README.

3. JaCoCo com meta mínima

```
<plugin>
  <groupId>org.jacoco</groupId>
  <artifactId>jacoco-maven-plugin</artifactId>
  <version>0.8.15</version>
  <executions>
    <execution>
      <goals><goal>prepare-agent</goal></goals>
    </execution>
    <execution>
      <id>report</id>
      <phase>test</phase>
      <goals><goal>report</goal></goals>
    </execution>
    <execution>
      <id>check</id>
      <goals><goal>check</goal></goals>
      <configuration>
        <rules>
          <rule>
            <limits>
              <!-- comece com a sua linha de base medida, nao invente um numero -->
              <limit><counter>LINE</counter><minimum>0.60</minimum></limit>
            </limits>
          </rule>
        </rules>
      </configuration>
    </execution>
  </executions>
</plugin>
```

4. Teste de carga com Gatling

```

import io.gatling.javaapi.core.*;
import io.gatling.javaapi.http.*;

import static io.gatling.javaapi.core.CoreDsl.*;
import static io.gatling.javaapi.http.HttpDsl.*;

public class ListarChamadosSimulation extends Simulation {

    HttpProtocolBuilder httpProtocol = http.baseUrl("http://localhost:8080");

    ScenarioBuilder cenario = scenario("Listar chamados sob carga")
        .exec(http("login")
            .post("/api/auth/login")
            .body(StringBody("{\"login\":\"usuario.teste\",\"senha\":\"senha123\"}")).asJson()
            .check(jmesPath("token").saveAs("token")))
        .exec(http("listar-chamados")
            .get("/api/chamado")
            .header("Authorization", "Bearer #{token}")
            .check(status().is(200)));

    {
        setUp(
            cenario.injectOpen(
                rampUsersPerSec(1).to(20).during(30) // sobe gradualmente ate 20 usuarios/s
            )
        ).protocols(httpProtocol)
        .assertions(
            global().responseTime().percentile(95).lt(500), // p95 abaixo de 500ms
            global().failedRequests().percent().lt(1.0) // menos de 1% de erro
        );
    }
}

```

Rode só contra o seu ambiente local ou de teste — nunca contra um ambiente de produção que você não controla.

5. IA aplicada a testes: gerar, depois desconfiar

O fluxo que vale para entrevista não é "pedir e aceitar", é "pedir, revisar, medir":

1. Peça à IA um rascunho de testes para uma classe pouco coberta (por exemplo, `TokenService`).
2. Passe o checklist: o teste falharia se a regra estivesse errada? Ele só espelha a implementação? Há mocks demais?
3. Rode o **PIT** (Fase 1) sobre essa classe.
4. Reforce as asserções até o mutation score subir, e anote o antes/depois.

```
// Exemplo do tipo de teste que uma IA costuma gerar – passa, mas nao prova nada:
@Test
void deveValidarToken() {
    String token = tokenService.generateToken(usuario);
    assertNotNull(token); // so confere que NAO e nulo
}

// Versao revisada, que de fato falharia se a regra quebrasse:
@Test
void deveValidarTokenGeradoParaOMesmoUsuario() {
    String token = tokenService.generateToken(usuario);

    String loginExtraido = tokenService.validateToken(token);

    assertThat(loginExtraido).isEqualTo(usuario.getLogin());
}

@Test
void deveRejeitarTokenAdulterado() {
    String token = tokenService.generateToken(usuario);
    String tokenAdulterado = token.substring(0, token.length() - 5) + "XXXXX";

    assertThat(tokenService.validateToken(tokenAdulterado)).isNull();
}
```

6. Aproveitando o módulo `tecnicos` para praticar TDD completo

O seu projeto já tem a entidade `Tecnico`, mas nenhum repository, service ou controller. É a chance perfeita de implementar um CRUD inteiro **testes primeiro**: comece pelo teste de `TecnicoService.criar(...)`, veja falhar (a classe nem existe), crie o mínimo, depois o repositório, depois o controller com `@WebMvcTest`.

7. Exercício desta fase

- ☐ Pipeline de CI rodando os testes unitários e de integração a cada push.
- ☐ JaCoCo com meta mínima baseada na sua cobertura real medida.
- ☐ Cenário Gatling para `GET /api/chamado` e `POST /api/auth/login`.
- ☐ Um caso documentado de teste gerado por IA, revisado e melhorado com PIT.
- ☐ Módulo `tecnicos` implementado do zero, testes primeiro.

Fontes oficiais

- [GitHub Actions — Building and testing Java with Maven](#)
- [JaCoCo — Maven Plugin](#)
- [Gatling — Maven Plugin](#)

Fase 5 — Portfólio e entrevistas

1. O README é a sua vitrine

Recrutador técnico abre o repositório antes de te chamar para conversar. O README precisa provar, em meio minuto de leitura, que você sabe testar.

Estrategia de testes

Nivel	Ferramenta	Quantidade	Tempo
---	---	---	---
Unitario	JUnit 5 + Mockito	42	8s
Integracao	Spring Boot Test + Testcontainers	18	47s
Automacao de API	RestAssured	25	1min12s

Mutation score (PIT) nas classes de servico: 84%.

Como rodar

- `./mvnw test`` - testes unitarios
- `./mvnw verify`` - unitarios + integracao (sobe Testcontainers)
- `cd automation-tests && ./mvnw test -DBASE_URL=http://localhost:8080`` - automacao de API

Use **só números que você mediu de verdade** rodando o próprio projeto. Um número inventado, numa entrevista técnica, é fácil de derrubar com uma pergunta de acompanhamento.

2. Currículo: bullets com verbo, ferramenta e resultado

Modelo: **verbo de ação + o que + ferramenta + resultado mensurável**.

- "Implementei suite de testes unitarios e de integracao para API REST em Spring Boot (JUnit 5, Mockito, Testcontainers), elevando a cobertura de X% para Y% e o mutation score para Z%."
- "Automatizei testes de API com RestAssured cobrindo autenticacao JWT e os fluxos criticos de um sistema de chamados, reduzindo testes manuais de regressao."
- "Configurei pipeline de CI no GitHub Actions com banco MySQL via Testcontainers e quality gate de cobertura no JaCoCo."

Troque os "X", "Y", "Z" pelos números reais do seu `techdesk-chamados-api` depois de rodar a Fase 4.

3. Perguntas de entrevista baseadas no seu próprio projeto

Pergunta	Resposta-modelo curta
Qual a diferença entre <code>@WebMvcTest</code> e <code>@SpringBootTest</code> ?	<code>@WebMvcTest</code> sobe só a camada web com o service mockado; <code>@SpringBootTest</code> sobe o contexto inteiro. Uso o primeiro para contrato HTTP, o segundo só nos fluxos críticos completos.
Por que Testcontainers em vez de H2?	Meu projeto roda em MySQL em produção; H2 tem dialeto SQL diferente e pode esconder bugs de query que só aparecem no banco real.
Mock, stub, spy ou fake: qual a diferença?	Mock eu programo e verifico interações; stub só devolve uma resposta fixa; spy envolve um objeto real; fake é uma implementação simplificada mas funcional, como um repositório em memória.
Encontrou algum bug testando este projeto?	Sim: o <code>ChamadoService.listarTodos()</code> engolia uma exceção em vez de propagá-la, e dois endpoints PATCH devolviam 201 em vez de 200. Escrevi testes que expunham os dois antes de decidir a correção.
Como você reduz testes instáveis (flaky)?	Isolando dados por teste, evitando dependência de ordem, nunca usando <code>Thread.sleep</code> (uso Awaitility quando preciso esperar algo assíncrono), e rodando a suíte várias vezes seguidas antes de confiar nela.
Como você validaria um teste gerado por IA?	Pergunto se ele falharia caso a regra estivesse errada, se não está só espelhando a implementação, e confirmo com mutation testing (PIT) rodando sobre a classe.
O que é mutation testing e por que cobertura de linha não basta?	Cobertura de linha só mostra que a linha rodou. O PIT muda o código (um <code>></code> vira <code>>=</code>) e roda os testes: se nenhum falhar, o teste não estava realmente verificando aquela lógica.

4. Um post curto para o LinkedIn

Passei as últimas semanas revisando testes em um projeto Java + Spring Boot que eu já tinha. Encontrei um bug real de exceção engolida só escrevendo os testes unitários do zero, e usei mutation testing (PIT) para confirmar que os testes realmente pegavam esse tipo de problema. Também testei a camada de segurança (JWT) e montei uma suíte de automação de API com RestAssured. Ficou no repo: [\[link\]](#). #Java #SpringBoot #Testes

5. Checklist final antes de aplicar para vagas

- ☐ README com a seção "Estratégia de testes" e números reais medidos.
- ☐ Pipeline de CI verde e público.
- ☐ Bullets do currículo com números do seu próprio projeto, não genéricos.
- ☐ Treinou em voz alta as 7 perguntas acima, sem ler a resposta.
- ☐ Aplicou para pelo menos 5 vagas citando Java + testes nesta semana.

Sobre as fontes desta fase

Esta fase não depende de documentação de uma ferramenta específica — ela usa as práticas de "Estratégia de testes" no README recomendadas pela própria comunidade Spring (mesmos padrões citados nas fontes das Fases 1 a 4) e boas práticas gerais de currículo técnico. Os números do quadro de exemplo são ilustrativos: substitua pelos que você medir de verdade no seu projeto.